SecureIncident: Security Design, Authorization and Vulnerability Analysis

Software Security | Assignment 1 | 18 October 2026
Project repository: github.com/MiguelM20/secureincident
Implementation: Python 3.12, FastAPI, SQLite, Docker Compose and Jenkins

1. Purpose, scope and architecture

SecureIncident is a small incident-management application designed to explore how security guarantees survive interactions between separate software components. An ordinary analyst creates, views and updates incidents, while an administrator receives additional oversight and reassignment permissions. Analysts also generate incident reports. Cases and reports belong to users, making unauthorized access observable with synthetic data. The project deliberately excludes registration, email, cloud hosting and a graphical interface, because those features would increase development effort without improving the assignment's main security experiment.

Three FastAPI applications run as independent processes and, in the final local deployment, as separate Docker containers. The **Gateway** accepts client requests, verifies user tokens and issues delegated operation grants. The **Incident Service** owns the SQLite database and enforces authorization against stored resource ownership. The **Report Service** performs report jobs using an independently installed Python formatter package. SQLite and the formatter are supporting dependencies, not additional application components. This separation deliberately creates interfaces where a service's technical access might be confused with the user's legitimate authority.

The main protected entities are incidents (owner, description, severity and status) and reports (associated incident, requesting user and generated text). Seeded accounts are Alice and Bob, both analysts, and one administrator. Users work through the Gateway; the Incident and Report services have no published host ports. Internal calls still require verified signed grants rather than assuming the Docker network itself establishes trust. The Report Service receives neither the Gateway's signing private key nor direct access to the Incident Service database.

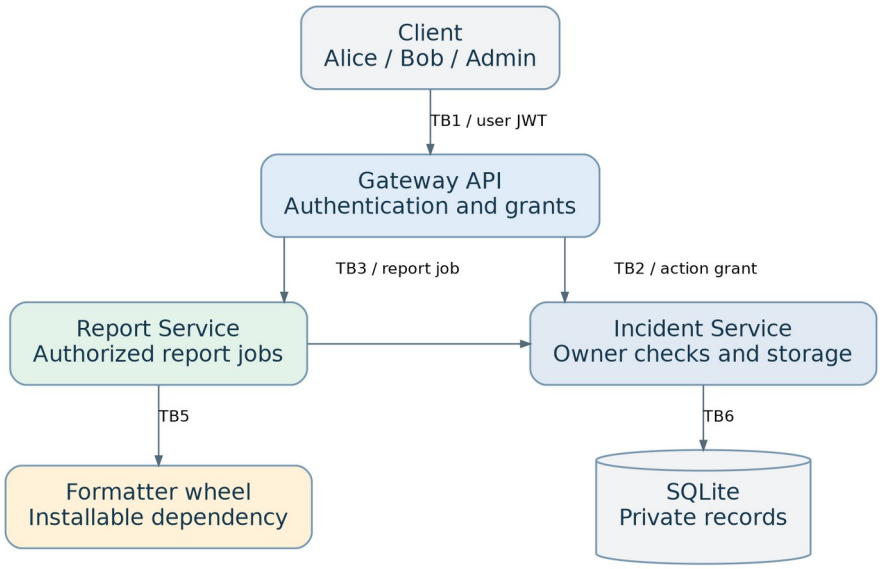


Figure 1. SecureIncident components and principal trust boundaries

Table 1. Components and trust boundaries

Boundary	Caller and receiver	Required check
TB1	User to Gateway	Login, signed user token and input validation
TB2	Gateway to Incident Service	JWT audience/scope plus stored owner/role
TB3	Gateway to Report Service	Signed, case-specific report job
TB4	Report Service to Incident Service	Original-user delegated grant and resource ownership
TB5	Report Service to formatter/file system	Declared dependency, content and path validation
TB6	Incident Service to SQLite	Parameterized SQL and private storage

The basic permission rule is deliberately simple: an analyst can act on cases whose stored `owner_id` equals the user identity in the verified grant, while an administrator can act across users within the operations explicitly provided. Ownership is assigned by the server when a case is created, and a normal status update cannot silently reassign it. Separate administrative endpoints make privilege-sensitive actions easier to review and test. The functional scope includes authentication, incident creation, listing and updating, report creation and retrieval, administrator reassignment, and an administrator-only overview. This exceeds the requirement for three useful operations. Report generation intentionally uses all three application processes. Docker Compose provides a reproducible demonstration with separate service images, networks, key volumes and Incident Service storage. Requirements and access policy, architecture and the data-flow diagram document the initial boundaries.

2. Identity, authority, ownership and data flow

The Incident Service stores salted password hashes using Argon2 through a maintained library. At login, the Gateway sends the supplied credentials to a restricted internal verification endpoint using a short-lived service grant. Successful verification returns the stored user identifier and role, not the password hash. The Gateway issues a signed RS256 user JWT valid for approximately 30 minutes. It verifies that token for subsequent public requests, including issuer, audience, signature and expiration. User-controlled request fields cannot assign a different identity or role.

For a protected action, the Gateway issues a fresh delegated JWT with an approximately two-minute lifetime. Its claims specify the original user (`sub`), role, target service (`aud`), requested action (`scope`), relevant incident identifier, and temporal validity. The Gateway alone holds the private signing key. The Incident and Report services verify the public key and appropriate claims. Crucially, a signed token is not treated as proof of resource ownership: the Incident Service loads the incident's stored owner and compares it with the authenticated subject, unless the stored role explicitly permits administrator access.

Consider Alice's request to generate a report for incident 101. Her bearer token arrives at the Gateway, which creates a report-job grant for the Report Service and separate `incident:read` and `report:create` grants restricted to incident 101. The grants share Alice's identity and a request correlation identifier. The Report Service validates their consistency, requests incident 101 from the

Incident Service, invokes the installable incident_formatter, and submits the resulting content to the Incident Service for storage. The resource service checks authorization on both the read and the write. The final response includes the saved report only after these operations succeed. If Alice attempts the identical workflow with Bob's incident 102, the resource lookup is denied and no report should be stored. This sequence diagram represents the authorized path.

Table 2. Access-control policy (selected operations)

Operation	Analyst on own case	Analyst on another case	Administrator
Create, read or update case	Allow	Deny	Allow
Generate or retrieve case report	Allow	Deny	Allow
Reassign ownership	Deny	Deny	Allow
View global overview	Deny	Deny	Allow

3. Secure design principles and threat modelling

Least privilege shaped both user permissions and deployment. Ordinary analysts access only their own incidents; administrator-only reassignment is a separate endpoint. The Report Service receives operation-specific, case-bound grants rather than database credentials or an administrator's ambient privileges. Docker gives only the Gateway access to the private signing key. The two resource-related services are internal, while the Gateway alone publishes a loopback port. Tests attempting a different incident ID or a mismatched job grant demonstrate the restrictions.

Complete mediation means authorization is checked whenever a protected resource is accessed, not merely when the user first logs in. The Incident Service checks signature, audience, operation scope, incident binding, current user role and the stored incident owner. It performs this mediation for requests arriving directly from the Gateway and indirectly through report generation. Negative tests verify that Alice cannot read, update or generate a report for Bob's incident, even while Alice has a valid login token.

Fail-safe defaults means missing proof of authority results in denial. The system rejects invalid, expired or wrong-audience grants. Unknown incidents and cases owned by another analyst are reported as not found, reducing disclosure of resource existence. Unsupported filenames and malformed requests are rejected before storage changes. Report generation does not persist an unauthorized result. A failed downstream request is treated as an error, not permission to continue. Tests check denied responses and lack of unintended state changes.

Threat modelling started from the assets: private incident descriptions, report content, password hashes, signed grants, the signing key, SQLite storage and the formatter dependency. A STRIDE review considered token spoofing, incident tampering, weak action traceability, cross-user information disclosure, resource exhaustion and privilege escalation through service composition. The highest-priority risks were cross-user reads and writes, inappropriate privileged delegation and signing-key compromise. This prioritization followed asset impact and reachable interfaces rather than an unsupported numeric risk score. The asset register and STRIDE matrix map threats to controls, tests and lifecycle gates.

4. Four planted vulnerabilities: demonstration and remediation

All experiments used local pytest fixtures, synthetic accounts and temporary databases or files. A dedicated Git branch preserved each vulnerable state, followed by a corrective commit. The final `main` branch contains the repaired code and the merged experiment evidence. Successful exploit tests are disabled during normal release testing, so a green pipeline never depends on vulnerabilities remaining exploitable.

4.1 SQL injection: CWE-89

The incident-search function originally uses parameterized LIKE conditions together with an analyst ownership predicate. The planted flaw interpolated the search string directly into SQL. Alice entered a crafted search value ending the quoted expression and adding a true condition. Because SQL interpreted her input as query structure rather than data, the result included both Alice's incident 101 and Bob's incident 102. This violated confidentiality despite valid authentication. The corrective implementation binds both search patterns as parameters and retains the stored-owner restriction. The isolated exploit printed `Injected search exposed incident IDs: [101, 102]`; the restored regression test passed. Vulnerable commit, fix commit and test evidence preserve this sequence.

4.2 Missing authorization: CWE-862

A second experiment removed the owner check from the Incident Service's incident-update operation while retaining signed identity and action scope checks. Alice then submitted an update for Bob's incident 102. The service accepted the write and changed its status from `open` to `resolved`, returning HTTP 200. Authentication had succeeded, but no decision linked Alice to ownership of the targeted record. Restoring the `permitted_incident` check before the update prevented the modification; the regression test passed. This is an integrity failure at the direct resource-operation boundary, not a token-forgery attack. Vulnerable commit, fix commit and evidence document the result.

4.3 Path traversal: CWE-22

Report generation accepts an optional text-report filename. The planted flaw removed basename restrictions, the random filename prefix and path-containment verification. An input containing `../` resolved outside the designated report working directory, and the test confirmed that synthetic report content was written outside the intended location within its private temporary fixture. The impact was unauthorized file placement within the process's effective file permissions, not a demonstrated host-system compromise. The fix validates an allowed `.txt` basename, applies a random prefix, resolves the final path, checks containment and uses exclusive file creation. A negative test subsequently rejected the unsafe filename without writing a report. Vulnerable commit, fix commit and evidence show the transition.

4.4 Cross-service confused deputy

The composition vulnerability was independent of the missing-authorization flaw. The Incident Service's stored-owner check remained active. Instead, the Gateway was intentionally modified to sign report-job, read and write grants using the seeded administrator identity, even when Alice initiated the request. The downstream services validated the signatures and scopes, but evaluated

authorization against the wrongly delegated administrator identity. Alice's direct request for Bob's incident returned HTTP 404, while her report request for the same incident returned HTTP 201 and exposed report content. The saved report attributed the job to the administrator. This demonstrated that a trusted service can wrongly amplify a caller's authority without any signature failure. The fix restores the actual requesting user's subject and role across all three grants; the cross-user report test then denied access and left no report behind. Vulnerable commit, fix commit and evidence establish the contrast.

5. SSDLC, CI/CD and release decisions

An initial four-phase SSDLC plan was committed before any feature code (initial commit). It mapped each asset and threat to a requirement, implementation activity, test and release-gate decision. Requirements defined user roles, trust boundaries and STRIDE threats. Implementation established independent services, dependency manifests and Docker images. Testing preserved exploit/fix histories and authorization regressions. The release/operation phase describes production controls rather than claiming the local demonstration is production-ready.

Jenkins LTS runs locally on macOS, with Docker Desktop providing the container engine. Its multibranch pipeline reads the repository's Jenkinsfile. Stages validate pinned direct dependencies, install the application, build the formatter wheel, run `pip check`, compile Python, execute Bandit static analysis and pip-audit dependency analysis, run pytest, and launch a fresh `secureincident-ci` Docker Compose project. The CI environment uses generated synthetic passwords, a separate key pair, fresh database volumes and host port 18080. After the HTTP smoke tests, Jenkins archives scan output, JUnit XML, Docker status/logs and the wheel, then removes its disposable containers and volumes.

The CI result is also a useful example of an SSDLC gate decision rather than a decorative scan. The first dependency-failure log identified the installed versions and their advisories; the gate stopped tests and Docker execution instead of reporting a misleading success. The changes were committed as an auditable dependency upgrade, and the tests and scanner were rerun together. This preserved the distinction between fixing a reported problem and merely hiding a warning. Release gates reject invalid direct dependency declarations, build errors, failed functional or authorization tests, high-severity/high-confidence Bandit findings and known dependency advisories. One earlier pipeline run failed when pip-audit found 26 advisories across PyJWT, pytest and Starlette. Dependencies were upgraded, without disabling the gate, and subsequent builds passed. The final saved pip-audit result reports no known vulnerabilities in the scanned environment. Bandit produced non-blocking lower-severity findings, rather than a completely empty report. Jenkins Build #4 succeeded after merging the experiments; its screenshot and archived reports are retained as evidence.

The Jenkins instance is reachable only on localhost, so direct GitHub webhooks are not available without additional network configuration. Multibranch indexing or periodic scans provide the intended mechanism for detecting new branch and pull-request revisions, but this project does not claim a separately verified immediate webhook-triggered PR run. This operational limitation should be distinguished from the successful manual CI build.

6. Verification, limitations and production considerations

The secure code passed **39 automated tests**, with four intentionally gated exploit tests skipped during the full local run. The Jenkins JUnit evidence for the earlier secure build records 39 tests with zero failures. The fresh Docker CI smoke checks returned the expected HTTP statuses: 200 for an owned incident, 404 for another analyst's case, 403 for an administrative action, 201 for authorized report generation, 422 for an unsafe filename and 200 for retrieval. All three CI containers reported healthy. These observations support the tested guarantees, but cannot prove the absence of all vulnerabilities.

Language/runtime features and established libraries provided parameterized database access, structured request validation, Argon2 password hashing and standards-based JWT verification. Application logic supplied the owner comparisons and job-to-user delegation policy. Deployment configuration supplied separate processes, a private database volume, limited key distribution, restricted container privileges and network segmentation. No single layer replaces the others.

The most important residual assumption concerns the integrity of the Gateway, because the cross-service experiment demonstrated that a legitimate signing key can still create improperly privileged grants when application logic chooses the wrong subject. Cryptographic verification alone cannot detect this semantic mistake. It requires trustworthy code, correctly scoped delegation and end-to-end authorization tests. A production release would need TLS for external and internal traffic, stronger service identity where appropriate, managed secrets and key rotation, audit logging with protected retention, rate limits, monitoring, backups, controlled image/dependency provenance, and reviewed rollout and recovery procedures. Reproducible installation presently pins direct dependencies, but transitives and base images would require stronger locking and digest pinning. The optional malicious helper-package supply-chain attack was not performed. The submitted project demonstrates narrow authority and corrected cross-service composition with synthetic data, while acknowledging these operational limitations.

References and evidence index

1. Repository and commit history.
2. Requirements and access-control matrix.
3. Architecture, interface boundaries and sequence.
4. STRIDE threat model and asset register.
5. Vulnerability experiment documentation and saved logs.
6. Jenkinsfile, pytest JUnit, Bandit, pip-audit and Docker smoke output.